BATCH 4

Tool-by-Tool Integration Map

Dragonfruit Media × AIGenX

PART 2 OF 2 — PRODUCTION, CREATIVE & OPERATIONS TOOLS

March 2026 | Confidential

Covering: Frame.io, Dropbox, YouTube, Make/Zapier, Descript, Adobe Suite, AI Creative Tools, QuickBooks, and Operations Stack

7. Tool 4: Frame.io

Current Spend	$1,095/mo ($13,140/yr) — Pro Plan
Users	10 members (CD, Editor, ECD, PPM, Producer, PC)
Pipeline Phases	Onboarding (Style Edit), Post-Production (all review cycles)
Integration Method	API REST v2 + WEBHOOK Comment/Review events
MCP Server	❌ No official MCP — custom Function Tools built
Agent Consumers	PPM Agent (ED-1 QC, ED-5 Sentiment), CD Agent (review), PC Agent (publish prep)

7.1 Frame.io's Critical Role

Frame.io is the review hub for every video DFM produces. Every edit version (V1 through V4+) passes through Frame.io for timestamped feedback from PPMs, CDs, ECDs, and clients. This makes it the richest source of **qualitative production data** in the entire pipeline.

7.2 Custom Function Tools (API Wrappers)

```
# Frame.io Function Tools for OpenAI Agents SDK
# File: integrations/frameio_tools.py

import httpx
from agents import function_tool

FRAMEIO_BASE = "https://api.frame.io/v2"

@function_tool
async def get_asset_comments(asset_id: str) -> dict:
    """Retrieve all timestamped comments on a Frame.io asset (video).
    Used by: PPM Agent (ED-1 QC Checker, ED-5 Sentiment Analyzer)

    Args:
        asset_id: The Frame.io asset ID for the video

    Returns:
        List of comments with timestamps, authors, and text
    """
    async with httpx.AsyncClient() as client:
        response = await client.get(
            f"{FRAMEIO_BASE}/assets/{asset_id}/comments",
            headers={"Authorization": f"Bearer {FRAMEIO_TOKEN}"}
        )
        comments = response.json()
        return {
            "asset_id": asset_id,
            "comment_count": len(comments),
            "comments": [
```

```
                {
                    "author": c["owner"]["name"],
                    "timestamp": c.get("timestamp"),  # Video timestamp
                    "text": c["text"],
                    "created_at": c["inserted_at"],
                    "completed": c.get("completed", False)
                }
                for c in comments
            ]
        }

@function_tool
async def get_review_status(project_id: str, asset_name: str) -> dict:
    """Check review/approval status of a video asset.
    Used by: PC Agent (publish prep), Producer Agent (timeline tracking)

    Args:
        project_id: Frame.io project ID
        asset_name: Name of the video asset to check
    """
    async with httpx.AsyncClient() as client:
        # Search for asset in project
        assets = await client.get(
            f"{FRAMEIO_BASE}/projects/{project_id}/assets",
            headers={"Authorization": f"Bearer {FRAMEIO_TOKEN}"},
            params={"name": asset_name}
        )
        asset = assets.json()[0]

        return {
            "asset_name": asset["name"],
            "status": asset.get("label", "none"),
            "version": asset.get("version", 1),
            "comment_count": asset.get("comment_count", 0),
            "unresolved_comments": count_unresolved(asset["id"]),
            "last_activity": asset["updated_at"]
        }

@function_tool
async def download_asset_for_qc(asset_id: str) -> dict:
    """Get download URL for a Frame.io asset for automated QC analysis.
    Used by: PPM Agent (ED-1 Automated Video QC Checker)

    Args:
        asset_id: The Frame.io asset ID
    """
    async with httpx.AsyncClient() as client:
        response = await client.get(
            f"{FRAMEIO_BASE}/assets/{asset_id}",
            headers={"Authorization": f"Bearer {FRAMEIO_TOKEN}"}
        )
        asset = response.json()
        return {
            "download_url": asset["original"],
            "name": asset["name"],
            "duration": asset.get("duration"),
            "resolution": f"{asset.get('width')}x{asset.get('height')}",
```

```
            "codec": asset.get("codec"),
            "filesize_mb": round(asset.get("filesize", 0) / 1048576, 1)
        }

@function_tool
async def create_review_link(asset_id: str, reviewer_email: str,
                             expires_days: int = 7) -> dict:
    """Create a shareable review link for client review.
    Used by: PC Agent (client review step)

    Args:
        asset_id: Frame.io asset ID
        reviewer_email: Email of the reviewer
        expires_days: Days until link expires
    """
    async with httpx.AsyncClient() as client:
        response = await client.post(
            f"{FRAMEIO_BASE}/assets/{asset_id}/review_links",
            headers={"Authorization": f"Bearer {FRAMEIO_TOKEN}"},
            json={
                "name": f"Review - {reviewer_email}",
                "allow_approvals": True,
                "notify_on_comment": True,
                "expires_at": (datetime.now() +
timedelta(days=expires_days)).isoformat()
            }
        )
        return response.json()
```

7.3 Webhook Event Handlers

```
# Frame.io Webhook Handler
# File: webhooks/frameio_handler.py

FRAMEIO_EVENT_ROUTING = {
    "comment.created": {
        "primary_agent": "ppm_agent",
        "actions": [
            "run_sentiment_analysis",      # ED-5
            "check_if_revision_needed",
            "route_to_editor_slack"
        ]
    },
    "asset.label_updated": {
        "primary_agent": "pc_agent",
        "conditions": {
            "label:approved": [
                "update_clickup_status",
                "notify_publish_prep",
                "trigger_export_download"
            ],
            "label:needs_review": [
                "notify_next_reviewer",
                "update_clickup_status"
            ]
```

```
        }
    },
    "asset.uploaded": {
        "primary_agent": "pc_agent",
        "actions": [
            "validate_upload_specs",
            "link_to_clickup_task",
            "notify_reviewer_queue"
        ]
    },
    "review_link.opened": {
        "primary_agent": "producer_agent",
        "actions": [
            "log_client_review_start",
            "start_review_timer"
        ]
    }
}
```

7.4 ED-1 QC Checker — Frame.io Integration Flow

The automated QC checker is one of the highest-impact Frame.io integrations. Here's the complete flow:

Step	Action	Integration
1	Editor uploads V1/V2 to Frame.io and updates ClickUp status	WEBHOOK asset.uploaded
2	Webhook triggers PPM Agent → downloads asset via API	API download_asset_for_qc
3	PPM Agent runs QC checks: intro/outro, audio levels, color, captions, export settings	CUSTOM ML pipeline
4	Agent reads client-specific checklist from Notion SOP	MCP Notion
5	QC report generated with timestamps and severity ratings	CUSTOM Report generator
6	If issues found: posts timestamped comments on Frame.io asset	API create_comment
7	Notifies PPM via Slack with summary + link to Frame.io	MCP Slack
8	PPM reviews AI QC report, approves/modifies, then sends to editor	HITL Gate

> **Frame.io Integration ROI:** PPMs currently spend 28% of capacity on video review/QC. Automated pre-screening catches 60-70% of technical issues before human review, reducing revision cycles from avg 3.2 to target 2.1 per video. Across 54 editors producing ~200 videos/month, this saves an estimated 80+ hours/month of revision time.

8. Tool 5: Dropbox

Current Spend	$768/mo ($9,216/yr) — Dropbox Advanced
Users	Editor, PC, Publisher
Pipeline Phases	Post-Production (footage), Pre-Production (voice notes), Production (raw uploads)
Integration Method	API REST v2 + MAKE File transfer scenarios

8.1 Dropbox in the Pipeline

Dropbox is the file storage backbone. Raw footage flows in from clients, editors download it for editing, and final exports are uploaded for publishing. The key automation opportunities are around the **footage ingest pipeline** and **publish prep file management**.

8.2 Function Tools

```
# Dropbox Function Tools
# File: integrations/dropbox_tools.py

@function_tool
async def check_footage_upload(client_folder: str) -> dict:
    """Check if new footage has been uploaded to a client's Dropbox folder.
    Used by: PC Agent (footage QC step)

    Args:
        client_folder: Path to client's footage folder in Dropbox
    """
    dbx = get_dropbox_client()
    entries = dbx.files_list_folder(client_folder).entries

    recent = [e for e in entries
              if hasattr(e, 'server_modified')
              and e.server_modified > datetime.now() - timedelta(hours=24)]

    return {
        "folder": client_folder,
        "new_files": len(recent),
        "files": [
            {
                "name": f.name,
                "size_gb": round(f.size / 1073741824, 2),
                "uploaded": f.server_modified.isoformat(),
                "path": f.path_display
            }
            for f in recent
        ],
        "total_folder_size_gb": sum(e.size for e in entries if hasattr(e, 'size')) /
1073741824
```

```
    }

@function_tool
async def create_shared_link(file_path: str) -> dict:
    """Create a shared link for a Dropbox file (for ClickUp task linking).
    Used by: PC Agent (linking footage to tasks)
    """
    dbx = get_dropbox_client()
    link = dbx.sharing_create_shared_link_with_settings(file_path)
    return {"url": link.url, "path": file_path}

@function_tool
async def organize_footage(source_path: str, client: str,
                           batch: str, content_type: str) -> dict:
    """Move and organize footage into the correct folder structure.
    Used by: PC Agent (footage management)

    Standard structure: /Clients/{client}/Batches/{batch}/{content_type}/Raw/
    """
    dbx = get_dropbox_client()
    target = f"/Clients/{client}/Batches/{batch}/{content_type}/Raw"

    # Create target folder if needed
    try:
        dbx.files_create_folder_v2(target)
    except:
        pass  # Folder exists

    # Move files
    entries = dbx.files_list_folder(source_path).entries
    moved = []
    for entry in entries:
        if is_video_file(entry.name):
            dbx.files_move_v2(entry.path_display, f"{target}/{entry.name}")
            moved.append(entry.name)

    return {
        "files_moved": len(moved),
        "destination": target,
        "files": moved
    }

@function_tool
async def upload_final_export(local_path: str, client: str,
                              video_title: str) -> dict:
    """Upload final approved export to Dropbox for publishing.
    Used by: PC Agent (publish prep step)

    Standard path: /Clients/{client}/Finals/{video_title}/
    """
    dbx = get_dropbox_client()
    target = f"/Clients/{client}/Finals/{video_title}"

    dbx.files_create_folder_v2(target)

    with open(local_path, "rb") as f:
        result = dbx.files_upload(
```

```
            f.read(),
            f"{target}/{os.path.basename(local_path)}",
            mode=dropbox.files.WriteMode.overwrite
        )

    link = dbx.sharing_create_shared_link_with_settings(result.path_display)

    return {
        "uploaded": result.name,
        "path": result.path_display,
        "size_gb": round(result.size / 1073741824, 2),
        "share_link": link.url
    }
```

8.3 Automated Footage Ingest Pipeline

Step	Current (Manual)	Automated	Integration
1	Client uploads footage to Dropbox, shares link via Slack	Dropbox webhook detects new upload in client folder	WEBHOOK
2	YPC manually confirms correct link and footage	PC Agent validates: file count, format, size, naming convention	API
3	YPC manually moves to correct folder	PC Agent auto-organizes into standard folder structure	API
4	YPC manually adds link to ClickUp task	PC Agent creates shared link → updates ClickUp task custom field	API + MCP
5	Editor manually downloads footage	Notification sent to assigned editor with direct download link	MCP Slack

9. Tool 6: YouTube Data API

Current Spend	$83/mo (YouTube TV/Services) + API is free (quota-based)
Integration Method	API YouTube Data API v3 + YouTube Analytics API
Agent Consumers	CD Agent (CD-1, CD-2, CD-3), PC Agent (publishing), Producer Agent (analytics)

9.1 YouTube API — Two Distinct Use Cases

Use Case	API	Agent	Wish List Tool
Competitor Intelligence	YouTube Data API v3 — search, videos, channels, captions	CD Agent (CD-2 Competitor Scraper)	CD-2: Competitor Script Scraper + Pattern Tracker
Performance Analytics	YouTube Analytics API — retention curves, demographics, traffic sources	CD Agent (CD-1 Retention Scoring), Producer Agent	CD-1: Predictive Retention Scoring Tool
Publishing	YouTube Data API v3 — videos.insert, thumbnails.set	PC Agent (publish pipeline)	Automated publish prep
A/B Testing	YouTube Data API v3 — thumbnails.set, videos.update	CD Agent (split test monitoring)	Post-publication monitoring automation

9.2 CD-2 Competitor Scraper — YouTube Integration

```
# CD-2: Competitor Script Scraper
# File: agents/cd_tools/competitor_scraper.py

@function_tool
async def scrape_competitor_outliers(channel_ids: list,
                                     lookback_days: int = 7) -> dict:
    """Monitor competitor channels and identify outlier videos.
    Runs weekly via scheduled trigger.

    Args:
        channel_ids: List of YouTube channel IDs to monitor
        lookback_days: How far back to look for new videos
    """
    youtube = build("youtube", "v3", developerKey=YT_API_KEY)

    outliers = []
    for channel_id in channel_ids:
        # Get channel's recent videos
        search = youtube.search().list(
            channelId=channel_id,
            type="video",
            order="date",
            publishedAfter=(datetime.now() -
```

```
timedelta(days=lookback_days)).isoformat() + "Z",
            part="snippet",
            maxResults=50
        ).execute()

        video_ids = [item["id"]["videoId"] for item in search["items"]]

        # Get video statistics
        stats = youtube.videos().list(
            id=",".join(video_ids),
            part="statistics,snippet,contentDetails"
        ).execute()

        # Get channel baseline (avg views for last 30 videos)
        baseline = await get_channel_baseline(youtube, channel_id)

        # Identify outliers (>2x baseline views)
        for video in stats["items"]:
            views = int(video["statistics"]["viewCount"])
            if views > baseline * 2:
                # Get captions/transcript
                transcript = await get_video_transcript(video["id"])

                outliers.append({
                    "channel": video["snippet"]["channelTitle"],
                    "title": video["snippet"]["title"],
                    "views": views,
                    "baseline_multiple": round(views / baseline, 1),
                    "published": video["snippet"]["publishedAt"],
                    "duration": video["contentDetails"]["duration"],
                    "thumbnail_url": video["snippet"]["thumbnails"]["maxres"]
["url"],
                    "transcript_preview": transcript[:500] if transcript else None,
                    "hook_analysis": None  # Filled by CD Agent reasoning
                })

    return {
        "period": f"Last {lookback_days} days",
        "channels_monitored": len(channel_ids),
        "outliers_found": len(outliers),
        "outliers": outliers
    }
```

9.3 Publishing Pipeline — YouTube Integration

```
# YouTube Publishing Tools
# File: integrations/youtube_tools.py

@function_tool
async def prepare_youtube_upload(video_path: str, metadata: dict) -> dict:
    """Prepare and execute YouTube video upload with full metadata.
    Used by: PC Agent (publish prep pipeline)

    Args:
        video_path: Path to the final export file
```

```
        metadata: Dict with title, description, tags, category,
                  thumbnail_path, schedule_time, privacy
    """
    youtube = get_authenticated_youtube_client(metadata["channel_id"])

    body = {
        "snippet": {
            "title": metadata["title"],
            "description": metadata["description"],
            "tags": metadata["tags"],
            "categoryId": metadata.get("category_id", "22")
        },
        "status": {
            "privacyStatus": "private",  # Always private first
            "publishAt": metadata.get("schedule_time"),
            "selfDeclaredMadeForKids": False
        }
    }

    # Upload video
    media = MediaFileUpload(video_path, resumable=True)
    request = youtube.videos().insert(
        part="snippet,status",
        body=body,
        media_body=media
    )

    response = None
    while response is None:
        status, response = request.next_chunk()

    video_id = response["id"]

    # Set thumbnail
    if metadata.get("thumbnail_path"):
        youtube.thumbnails().set(
            videoId=video_id,
            media_body=MediaFileUpload(metadata["thumbnail_path"])
        ).execute()

    return {
        "video_id": video_id,
        "url": f"https://youtube.com/watch?v={video_id}",
        "status": "private_scheduled",
        "scheduled_for": metadata.get("schedule_time"),
        "hitl_note": "Video uploaded as PRIVATE. Publisher must confirm before going
live."
    }
```

10. Tool 7: Make & Zapier

Current Spend	Make: $559.58/mo ($6,715/yr) — Teams Plan | Zapier: $133.50/mo ($1,602/yr)
Combined	$693/mo ($8,317/yr)
Integration Method	MAKE Preserved scenarios + WEBHOOK Agent-triggered

10.1 Strategy: Preserve, Extend, Don't Replace

> **Critical Design Decision:** We do NOT replace existing Make/Zapier scenarios. They work. They're tested. The team trusts them. Instead, agents **extend** these scenarios by: (1) providing better input data, (2) consuming their outputs for reasoning, and (3) triggering new scenarios for agent-initiated workflows.

10.2 Existing Scenarios — Inventory & Agent Touchpoints

#	Platform	Scenario Name	Trigger	Action	Agent Touchpoint
1	Make	Production Timeline Automation	New parent task in ClickUp	Creates timeline structure	Producer Agent provides richer task data via MCP before this fires
2	Make	Creation of Subtasks in Editing Batches	Webhook from ClickUp	Auto-names and creates subtasks	PC Agent validates completeness after creation
3	Make	Subtask Status → Ready to Start	Status change to "ready to start"	Sends webhook for Video/Short tasks	PC Agent (ED-2) intercepts webhook for capacity-based assignment
4	Make	Auto-Link Editor via Relationship	Edit V1 assigned	Links editor to Editor Database	PPM Agent reads this relationship for performance tracking
5	Make	Weekly Accumulator (×3)	Scheduled (weekly)	Aggregates editor database tracking	PPM Agent consumes accumulated data for pattern reports
6	Make	Active Video Tracker (×3)	Editor downloads footage	Old tracking automations	Candidate for deprecation — agent tracking is more comprehensive
7	Zapier	Missed Due Date Tracker	Subtask due date passes	Tracks missed deadlines	Producer Agent receives alerts, auto-cascades timeline adjustments

10.3 New Agent-Triggered Make Scenarios

```
# New Make scenarios triggered by agents via webhook
# File: integrations/make_triggers.py

NEW_MAKE_SCENARIOS = [
    {
        "name": "Agent → Async Standup Distribution",
        "trigger": "Agent webhook (daily 8am per timezone)",
        "action": "Compiles ClickUp data → formats standup → posts to pod channels",
        "agent": "PC Agent",
        "replaces": "Manual daily standup prep (saves 2h/day across PCs)"
    },
    {
        "name": "Agent → Client Sentiment Alert",
        "trigger": "Agent webhook (when sentiment drops below threshold)",
        "action": "Compiles sentiment data → creates Slack alert → creates ClickUp
task for Producer",
        "agent": "PPM Agent (ED-5)",
        "replaces": "No current equivalent — new capability"
    },
    {
        "name": "Agent → Weekly Competitor Digest",
        "trigger": "Agent webhook (weekly, Monday 6am)",
        "action": "Formats CD-2 outlier data → creates Notion page → posts summary
to #creative-intel",
        "agent": "CD Agent (CD-2)",
        "replaces": "Manual competitor research (saves 3-5h/week per CD)"
    },
    {
        "name": "Agent → Publish Prep Checklist",
        "trigger": "Agent webhook (when video status = approved)",
        "action": "Creates checklist in ClickUp → generates Dropbox links → notifies
publisher",
        "agent": "PC Agent",
        "replaces": "Manual publish prep coordination (saves 30min per video)"
    },
    {
        "name": "Agent → Monthly Report Scaffold",
        "trigger": "Agent webhook (1st of month)",
        "action": "Pulls YouTube analytics → creates Notion report template →
assigns to DA",
        "agent": "CD Agent",
        "replaces": "Manual report setup (saves 2h/month per client)"
    }
]
```

11. Tool 8: Descript

Current Spend	$70/mo ($840/yr) — Creator Plan
Users	Creative Director, Editor
Integration Method	NATIVE (Descript ↔ Premiere) + MAKE file triggers

11.1 Descript's Pipeline Role

Descript sits at the rough cut stage — editors upload footage, use transcript-based editing to cut down raw footage, then export XML to Adobe Premiere for full editing. The CD also reviews rough cuts in Descript.

11.2 Integration Approach

Descript has a limited API. Our integration is primarily through:

- **File-based triggers:** When an editor exports from Descript to a shared folder, a Make scenario detects the new file and updates the ClickUp task status
- **Transcript extraction:** Descript's transcript output is consumed by the CD Agent for rough cut review context
- **Native Premiere integration:** Preserved as-is — editors continue their existing Descript → Premiere workflow

> **ED-3 Rough Cut Acceleration:** While Descript handles transcript-based editing, the ED-3 tool goes further by using AI to auto-sync dialogue to script sections, remove silences, and produce a structured timeline. This runs *alongside* Descript, not replacing it — the output is an enhanced starting point that editors refine in Descript/Premiere.

12. Tool 9: Adobe Creative Suite

Current Spend	TBD (listed as pending in tool stack)
Products	Premiere Pro, After Effects, Photoshop (+ Firefly AI)
Integration Method	NATIVE — No direct agent integration

12.1 Why Adobe Stays Native

Adobe's creative tools are the editor's workspace. Agents don't enter this space — they **prepare inputs** and **process outputs**:

Adobe Tool	Agent Prepares (Input)	Agent Processes (Output)

Premiere Pro	Organized footage (Dropbox), script (Notion), roadmap (ClickUp), style guide (Notion)	Exported video → Frame.io for review → QC checker (ED-1)
After Effects	Roadmap feedback (ClickUp), graphics specs (Notion)	Exported roadmap → ClickUp for CD review
Photoshop	Thumbnail brief (Notion), packaging analysis (CD-3)	Thumbnail upload → ClickUp for CD review → YouTube for A/B test

12.2 Adobe Firefly — Survey-Identified Opportunity

50% of editors (23/46) already use Adobe Firefly. The survey shows it's used for frame extension, generative fill, and aspect ratio changes. No agent integration needed — this is a **tool provisioning** recommendation:

> **Recommendation:** Ensure all editors have access to Firefly features within their Adobe CC subscription. The 50% adoption rate without formal rollout suggests high organic demand. Formal training (per Batch 3 playbooks) would push adoption to 80%+.

13. Tool 10: AI Creative Tools (Survey-Identified)

The AI Tool Survey identified 20+ tools in active use across the team. These are **not agent-integrated** — they're individual productivity tools. Our role is to ensure proper provisioning and training.

Tool	Category	Adoption	Top Roles	Recommendation
ChatGPT	General AI	89% editors, 100% all others	All roles	✅ Already provisioned ($200/mo). Expand to team-wide paid access. 17 editors pay out of pocket.
Claude	Strategic AI	50% producers, 50% PPMs	Producers, PPMs	✅ Provision for Producers + PPMs. Powers CD-4 Knowledge Bases and agent SDK.
ElevenLabs	Audio AI	100% PPMs, 52% editors	PPMs, Editors	✅ Already provisioned ($22/mo). Consider team plan for PPMs.
Gemini	General AI	67% editors, 56% PCs	Editors, PCs	⚠️ Monitor. High adoption but overlaps with ChatGPT. Consider consolidation.
Freepik AI	Image Gen	57% editors	Editors, Designers	✅ Already provisioned ($100/mo across 5 accounts).
Higgsfield	Video Gen	28% editors, 20% CDs	Editors	🔴 #2 most requested tool. 11 editors want it. Evaluate for team license.
Google Veo	Video Gen	37% editors	Editors	⚠️ Free tier. Monitor quality improvements.
Kling AI	Video Gen	24% editors, 50% PPMs	Editors, PPMs	⚠️ Evaluate alongside Higgsfield for video gen standardization.
Runway	Video Gen	75% PPMs	PPMs	⚠️ PPM-specific. 2 PPMs requested it. Evaluate for PPM team license.
Midjourney	Image Gen	Low current	PPMs	⚠️ 2 PPMs requested. Evaluate vs. Freepik/ Firefly.
Wispr Flow	Voice-to-Text	Requested by CDs	CDs	🔴 CD-specific request. Voice-to-text for faster ideation. Evaluate.

14. Tool 11: QuickBooks Online (QBO)

Current Spend	$115/mo ($1,380/yr) — QuickBooks Plus
Integration Method	API QuickBooks Online API + MAKE
Wish List Tools	FA-1 (Automated Invoicing), FA-2 (Late Invoice Alerts), FA-3 (Monthly Reports), FA-5 (Financial Modelling)

14.1 Finance Agent — QBO Integration

```
# Finance Agent — QuickBooks Function Tools
# File: integrations/qbo_tools.py

@function_tool
async def create_invoice(client_name: str, line_items: list,
                         due_date: str) -> dict:
    """Create an invoice in QuickBooks Online.
    Used by: Finance Agent (FA-1)

    Args:
        client_name: Client display name in QBO
        line_items: List of {description, amount, quantity}
        due_date: Invoice due date (YYYY-MM-DD)
    """
    qbo = get_qbo_client()

    customer = qbo.query(f"SELECT * FROM Customer WHERE DisplayName =
'{client_name}'")

    invoice = Invoice()
    invoice.CustomerRef = {"value": customer[0].Id}
    invoice.DueDate = due_date
    invoice.Line = [
        {
            "Amount": item["amount"] * item["quantity"],
            "DetailType": "SalesItemLineDetail",
            "Description": item["description"],
            "SalesItemLineDetail": {
                "Qty": item["quantity"],
                "UnitPrice": item["amount"]
            }
        }
        for item in line_items
    ]

    result = invoice.save(qb=qbo)
    return {
        "invoice_id": result.Id,
        "total": float(result.TotalAmt),
        "due_date": due_date,
        "status": "sent" if auto_send else "draft"
```

```
    }

@function_tool
async def check_overdue_invoices() -> dict:
    """Check for overdue invoices across all clients.
    Used by: Finance Agent (FA-2) — runs daily
    """
    qbo = get_qbo_client()

    overdue = qbo.query(
        "SELECT * FROM Invoice WHERE DueDate < '{}' AND Balance > '0'"
        .format(datetime.now().strftime("%Y-%m-%d"))
    )

    return {
        "overdue_count": len(overdue),
        "total_overdue": sum(float(inv.Balance) for inv in overdue),
        "invoices": [
            {
                "client": inv.CustomerRef.name,
                "amount": float(inv.Balance),
                "due_date": inv.DueDate,
                "days_overdue": (datetime.now() - parse_date(inv.DueDate)).days,
                "escalation_tier": get_escalation_tier(inv)
            }
            for inv in overdue
        ]
    }

@function_tool
async def generate_margin_report(period: str) -> dict:
    """Generate monthly margin report by client and service line.
    Used by: Finance Agent (FA-3)

    Args:
        period: Report period (e.g., "2026-02")
    """
    qbo = get_qbo_client()

    # Pull revenue by client
    revenue = qbo.query(f"""
        SELECT CustomerRef, SUM(TotalAmt) as Revenue
        FROM Invoice
        WHERE TxnDate >= '{period}-01' AND TxnDate <= '{period}-28'
        GROUP BY CustomerRef
    """)

    # Pull costs (from cost model)
    costs = await get_cost_model_data(period)

    # Calculate margins
    report = []
    for client_rev in revenue:
        client_cost = costs.get(client_rev.CustomerRef.name, 0)
        margin = (float(client_rev.Revenue) - client_cost) /
float(client_rev.Revenue)
        report.append({
```

```
            "client": client_rev.CustomerRef.name,
            "revenue": float(client_rev.Revenue),
            "cost": client_cost,
            "margin": round(margin * 100, 1),
            "flag": "⚠️" if margin < 0.3 else "✅"
        })

    return {
        "period": period,
        "total_revenue": sum(r["revenue"] for r in report),
        "total_cost": sum(r["cost"] for r in report),
        "avg_margin": round(sum(r["margin"] for r in report) / len(report), 1),
        "clients": report
    }
```

15. Tools 12-15: Remaining Operations Stack

Tool	Spend	Integration	Agent Touchpoint
Nusii (Proposals)	$129/mo	WEBHOOK	Webhook on "contract signed" → triggers onboarding pipeline in Producer Agent
HelloSign (E-signatures)	$20/mo	API WEBHOOK	HR Agent (HR-3): auto-sends onboarding docs, tracks completion status
Zoom (Meetings)	$477/mo	API	Producer Agent: auto-schedules kickoff calls, pulls meeting transcripts via Grain integration
Grain (Meeting Notes)	$555/mo	API	Producer Agent: consumes meeting summaries for action item extraction → creates ClickUp tasks
Loom (Video Messages)	$144/mo	API	PPM Agent: consumes Loom transcripts for editor feedback context
Missive (Email)	$686/mo	API	Producer Agent (PR-2): monitors client email sentiment; Finance Agent (FA-2): sends invoice reminders
Google Workspace	$932/mo	API	Calendar API for scheduling; Sheets API for data analyst reports; Forms API for KO Survey intake
Notta (Transcription)	TBD	API	CD Agent: consumes voice note transcripts for script context; links to ClickUp tasks
1of10 / **ViewStats**	$129/mo combined	API (if available) / CUSTOM scraping	CD Agent (CD-2): competitor data source; supplements YouTube Data API

16. Master Integration Map — All Tools × All Agents

This matrix shows every agent-to-tool connection in the system:

Tool	Producer Agent	PC Agent	PPM Agent	CD Agent	Editor Support	Finance Agent	HR Agent
ClickUp	R/W (MCP)	R/W (MCP)	R/W (MCP)	R (MCP)	R (MCP)	—	—
Slack	R/W (MCP)	R/W (MCP)	R/W (MCP)	R/W (MCP)	R (MCP)	W (MCP)	R/W (MCP)
Notion	R/W (MCP)	R (MCP)	R (MCP)	R/W (MCP)	R (MCP)	—	R (MCP)
Frame.io	R (API)	R/W (API)	R/W (API)	R (API)	—	—	—
Dropbox	—	R/W (API)	—	—	R (API)	—	—
YouTube	R (API)	W (API)	—	R (API)	—	—	—
Make	Trigger (WH)	Trigger (WH)	Trigger (WH)	Trigger (WH)	—	Trigger (WH)	—
QBO	—	—	—	—	—	R/W (API)	—
Zoom	R/W (API)	—	—	—	—	—	—
Grain	R (API)	—	—	—	—	—	—
Missive	R (API)	—	—	—	—	W (API)	—
HelloSign	—	—	—	—	—	—	R/W (API)
Google WS	R/W (API)	R (API)	—	—	—	R (API)	—
Nusii	R (WH)	—	—				